

I. Problem Description

1. Pick an image of your choice. Apply all six style transfers available in <https://github.com/lengstrom/fast-style-transfer/> to this image. For this homework, print the original image and all six style-transferred images and hand them in.

II. Image Results

Environment: C:\AI\fast-style-transfer-master\fast-style-transfer-master



stata.jpg is from the original github file without transformed results. Then, downloading the ckpt files for six styles to transform stata.jpg to below distinct styles.

Terminal: python evaluate.py --checkpoint **goal**.ckpt --in-path stata.jpg --out-path **goal**.jpg

1. stata_la_muse.jpg:



2. stata_rain_princess.jpg:



3. stata_scream.jpg:



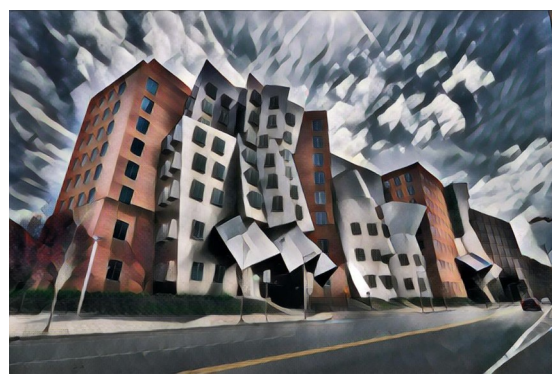
4. stata_wave.jpg:



5. stata_wreck.jpg:



6. stata_udnie.jpg:



III. Source code:

A little change for evaluate.py for the version compatible problem, rest of code unchanged.

```
from __future__ import print_function
import sys
sys.path.insert(0, 'src')
import transform, numpy as np, vgg, pdb, os
import scipy.misc
import tensorflow as tf

# 確保 tensorflow2.21 可以在 python3.9 下運作
if hasattr(tf, 'compat') and hasattr(tf.compat, 'v1'):
    import tensorflow.compat.v1 as tf
    tf.disable_v2_behavior()
else:
    pass
from utils import save_img, get_img, exists, list_files
from argparse import ArgumentParser
from collections import defaultdict
import time
import json
import subprocess
import numpy as np
from moviepy.video.io.VideoFileClip import VideoFileClip
import moviepy.video.io.ffmpeg_writer as ffmpeg_writer

BATCH_SIZE = 4
DEVICE = '/gpu:0'
```

```

def ffwd_video(path_in, path_out, checkpoint_dir, device_t='/gpu:0', batch_size=4):
    video_clip = VideoFileClip(path_in, audio=False)
    video_writer = ffmpeg_writer.FFMPEG_VideoWriter(path_out, video_clip.size, video_clip.fps,
    codec="libx264",
                                preset="medium", bitrate="2000k",
                                audiofile=path_in, threads=None,
                                ffmpeg_params=None)

    g = tf.Graph()
    soft_config = tf.compat.v1.ConfigProto(allow_soft_placement=True)
    soft_config.gpu_options.allow_growth = True
    with g.as_default(), g.device(device_t), \
        tf.compat.v1.Session(config=soft_config) as sess:
        batch_shape = (batch_size, video_clip.size[1], video_clip.size[0], 3)
        img_placeholder = tf.compat.v1.placeholder(tf.float32, shape=batch_shape,
            name='img_placeholder')

        preds = transform.net(img_placeholder)
        saver = tf.compat.v1.train.Saver()
        if os.path.isdir(checkpoint_dir):
            ckpt = tf.train.get_checkpoint_state(checkpoint_dir)
            if ckpt and ckpt.model_checkpoint_path:
                saver.restore(sess, ckpt.model_checkpoint_path)
            else:
                raise Exception("No checkpoint found...")
        else:
            saver.restore(sess, checkpoint_dir)

        X = np.zeros(batch_shape, dtype=np.float32)

        def style_and_write(count):
            for i in range(count, batch_size):
                X[i] = X[count - 1] # Use last frame to fill X
            _preds = sess.run(preds, feed_dict={img_placeholder: X})
            for i in range(0, count):
                video_writer.write_frame(np.clip(_preds[i], 0, 255).astype(np.uint8))

        frame_count = 0 # The frame count that written to X
        for frame in video_clip.iter_frames():
            X[frame_count] = frame
            frame_count += 1
            if frame_count == batch_size:
                style_and_write(frame_count)
                frame_count = 0

        if frame_count != 0:
            style_and_write(frame_count)

        video_writer.close()

# get img_shape

```

```

def fwd(data_in, paths_out, checkpoint_dir, device_t='/gpu:0', batch_size=4):
    assert len(paths_out) > 0
    is_paths = type(data_in[0]) == str
    if is_paths:
        assert len(data_in) == len(paths_out)
        img_shape = get_img(data_in[0]).shape
    else:
        # assert data_in.size[0] == len(paths_out)
        # img_shape = X[0].shape
        assert len(data_in) == len(paths_out)
        img_shape = data_in[0].shape

    g = tf.Graph()
    batch_size = min(len(paths_out), batch_size)
    curr_num = 0
    soft_config = tf.compat.v1.ConfigProto(allow_soft_placement=True)
    soft_config.gpu_options.allow_growth = True
    with g.as_default(), g.device(device_t), \
        tf.compat.v1.Session(config=soft_config) as sess:
        batch_shape = (batch_size,) + img_shape
        img_placeholder = tf.compat.v1.placeholder(tf.float32, shape=batch_shape,
                                                    name='img_placeholder')

        preds = transform.net(img_placeholder)
        saver = tf.compat.v1.train.Saver()
        if os.path.isdir(checkpoint_dir):
            ckpt = tf.train.get_checkpoint_state(checkpoint_dir)
            if ckpt and ckpt.model_checkpoint_path:
                saver.restore(sess, ckpt.model_checkpoint_path)
            else:
                raise Exception("No checkpoint found...")
        else:
            saver.restore(sess, checkpoint_dir)

    num_iters = int(len(paths_out)/batch_size)
    for i in range(num_iters):
        pos = i * batch_size
        curr_batch_out = paths_out[pos:pos+batch_size]
        if is_paths:
            curr_batch_in = data_in[pos:pos+batch_size]
            X = np.zeros(batch_shape, dtype=np.float32)
            for j, path_in in enumerate(curr_batch_in):
                img = get_img(path_in)
                assert img.shape == img_shape, \
                    'Images have different dimensions.' + \
                    'Resize images or use --allow-different-dimensions.'
                X[j] = img
            else:
                X = data_in[pos:pos+batch_size]

        _preds = sess.run(preds, feed_dict={img_placeholder:X})
        for j, path_out in enumerate(curr_batch_out):

```

```

        save_img(path_out, _preds[j])

    remaining_in = data_in[num_iters*batch_size:]
    remaining_out = paths_out[num_iters*batch_size:]
    if len(remaining_in) > 0:
        ffwd(remaining_in, remaining_out, checkpoint_dir,
              device_t=device_t, batch_size=1)

def ffwd_to_img(in_path, out_path, checkpoint_dir, device='/cpu:0'):
    paths_in, paths_out = [in_path], [out_path]
    ffwd(paths_in, paths_out, checkpoint_dir, batch_size=1, device_t=device)

def ffwd_different_dimensions(in_path, out_path, checkpoint_dir,
                              device_t=DEVICE, batch_size=4):
    in_path_of_shape = defaultdict(list)
    out_path_of_shape = defaultdict(list)
    for i in range(len(in_path)):
        in_image = in_path[i]
        out_image = out_path[i]
        shape = "%dx%dx%d" % get_img(in_image).shape
        in_path_of_shape[shape].append(in_image)
        out_path_of_shape[shape].append(out_image)
    for shape in in_path_of_shape:
        print('Processing images of shape %s' % shape)
        ffwd(in_path_of_shape[shape], out_path_of_shape[shape],
              checkpoint_dir, device_t, batch_size)

def build_parser():
    parser = ArgumentParser()
    parser.add_argument('--checkpoint', type=str,
                        dest='checkpoint_dir',
                        help='dir or .ckpt file to load checkpoint from',
                        metavar='CHECKPOINT', required=True)

    parser.add_argument('--in-path', type=str,
                        dest='in_path', help='dir or file to transform',
                        metavar='IN_PATH', required=True)

    help_out = 'destination (dir or file) of transformed file or files'
    parser.add_argument('--out-path', type=str,
                        dest='out_path', help=help_out, metavar='OUT_PATH',
                        required=True)

    parser.add_argument('--device', type=str,
                        dest='device', help='device to perform compute on',
                        metavar='DEVICE', default=DEVICE)

    parser.add_argument('--batch-size', type=int,
                        dest='batch_size', help='batch size for feedforwarding',
                        metavar='BATCH_SIZE', default=BATCH_SIZE)

    parser.add_argument('--allow-different-dimensions', action='store_true',

```

```

        dest='allow_different_dimensions',
        help='allow different image dimensions')

    return parser

def check_opts(opts):
    exists(opts.checkpoint_dir, 'Checkpoint not found!')
    exists(opts.in_path, 'In path not found!')
    if os.path.isdir(opts.out_path):
        exists(opts.out_path, 'out dir not found!')
        assert opts.batch_size > 0

def main():
    parser = build_parser()
    opts = parser.parse_args()
    check_opts(opts)

    if not os.path.isdir(opts.in_path):
        if os.path.exists(opts.out_path) and os.path.isdir(opts.out_path):
            out_path = \
                os.path.join(opts.out_path, os.path.basename(opts.in_path))
        else:
            out_path = opts.out_path

    ffwd_to_img(opts.in_path, out_path, opts.checkpoint_dir,
                device=opts.device)
    else:
        files = list_files(opts.in_path)
        full_in = [os.path.join(opts.in_path, x) for x in files]
        full_out = [os.path.join(opts.out_path, x) for x in files]
        if opts.allow_different_dimensions:
            ffwd_different_dimensions(full_in, full_out, opts.checkpoint_dir,
                                     device_t=opts.device, batch_size=opts.batch_size)
        else :
            ffwd(full_in, full_out, opts.checkpoint_dir, device_t=opts.device,
                 batch_size=opts.batch_size)

if __name__ == '__main__':
    main()

```

為 evaluate.py 的 source code 。